

Convolutional Neural Networks (CNN)

Course: Machine Learning | CSE 411
Green University of Bangladesh

Prepared by:

Md. Sharifur Rahman
Lecturer, Dept. of CSE
Green University of Bangladesh

Topics Covered:

1. CNN Architecture Overview
2. Input Layer & RGB Representation
3. Convolution Operation & Kernel
4. Zero-Padding & Stride
5. Output Dimension Calculation
6. Multi-Channel Convolution
7. Multiple Filters & Feature Maps
8. ReLU Activation Function
9. Flatten, Dense & Softmax Output
10. Complete End-to-End CNN Example

1. CNN Architecture Overview

A **Convolutional Neural Network (CNN)** is a specialised deep-learning architecture for grid-structured data such as images. CNNs exploit **local connectivity** and **weight sharing** via learnable filters, automatically detecting spatial hierarchies of features (edges → shapes → objects) with far fewer parameters than fully-connected networks.

Key Insight

Layer 1 → low-level features: edges, corners, colour blobs

Layer 2 → mid-level: textures, curves, simple shapes

Deeper layers → high-level: faces, wheels, entire objects

Stage	Components	Purpose
Feature Extraction	Conv + ReLU + Pooling (repeated)	Learn spatial patterns
Classification	Flatten → Dense → Softmax	Map to class probabilities

2. Input Layer — Image Representation

Every image is stored as a **3-D tensor: Width × Height × Channels**. An RGB colour image has **3 channels**; a grayscale image has 1. Each value is a pixel intensity in [0, 255].

Key Formula

Input Tensor Shape = Width x Height x Channels

Example: 128x128 colour image → 128 x 128 x 3 (49,152 values)

28x28 grayscale image → 28 x 28 x 1 (784 values)

3. Convolution Operation

The **convolution operation** slides a learnable **kernel** across the input. At each position, values are multiplied element-wise and summed, producing one entry in the output **feature map**.

$$O = (M - F + 2P) / S + 1$$

Symbol	Meaning	Typical Values
M	Input spatial size	28, 64, 224
F	Kernel size	3, 5, 7
P	Zero-padding	0, 1, 2
S	Stride	1 or 2
O	Output size	Computed

Example: 6×6 input, 3×3 kernel, P=0, S=1 → $O = (6-3+0)/1+1 = 4$

4. Cross-Correlation and Bias

CNNs use **cross-correlation** (kernel is not flipped before sliding). The network learns the appropriate weights regardless. A **bias** term b is added after the dot product: $y = w \cdot x + b$.

Bias

Bias is a single learnable value added after the dot product.

It lets the neuron shift its activation independently of the input.

Without bias, all outputs would be constrained through the origin.

5. Padding

Padding adds zeros around the input border. It preserves spatial dimensions ("same" padding) and ensures border pixels contribute equally.

Example (P=1, F=3, S=1, M=5): $O = (5-3+2)/1+1 = 5$

Same vs Valid Padding

"Valid" (P=0): output shrinks $\rightarrow O = M - F + 1$

"Same" (P=1 for F=3): output equals input size $\rightarrow O = M$

"Same" padding is standard in modern CNN architectures.

6. Output Dimension Calculation

Dimension	Formula
Output Height (OH)	$OH = \text{floor}((H + 2P - F) / S) + 1$
Output Width (OW)	$OW = \text{floor}((W + 2P - F) / S) + 1$
Output Depth (OD)	$OD = K$ (number of filters)

Example: Input $64 \times 64 \times 3$, $F=5$, $P=2$, $S=1$, $K=16 \rightarrow$ Output = **$64 \times 64 \times 16$**

7. Multi-Channel Convolution & Multiple Filters

For a C-channel input each kernel has depth C. The C channel results are **summed + bias** to produce one 2-D feature map. K kernels → output depth K.

$$y = (K_1 \blacksquare I_1) + \dots + (K_C \blacksquare I_C) + b$$

Multiple Filters

Each filter detects a different spatial pattern.

Filter 1 → horizontal edges Filter 2 → vertical edges

Filter 3 → diagonal gradients Filter K → abstract high-level patterns

8. ReLU Activation Function

After every convolution, **ReLU** is applied element-wise, setting all negatives to zero and passing positives unchanged.

$$\text{ReLU}(x) = \max(0, x)$$

Property	Detail
Fast	One threshold — no exponentials needed.
Sparse	Negatives $\rightarrow 0$; sparse, efficient representations.
No vanishing gradient	Gradient = 1 for positive inputs.
Fast convergence	Networks converge much faster than with sigmoid/tanh.

9. Flatten Layer, Dense Layer & Softmax Output

After the final convolutional and pooling layers have extracted rich spatial features, the network transitions from **feature extraction** to **classification**. This is done through three consecutive steps: Flatten → Dense (Fully Connected) → Softmax.

9.1 Flatten Layer

The **Flatten** layer reshapes the 3-D feature volume ($W \times H \times D$) produced by the last pooling layer into a **1-D vector**. No weights are learned; it is a pure reshape operation.

Formula:

$$\text{Flattened size} = W \times H \times D$$

Worked Example

Suppose the last pooling layer outputs a volume of shape $3 \times 3 \times 4$. The Flatten layer converts this into a vector of length:

$$3 \times 3 \times 4 = 36 \text{ values}$$

The 36 values are simply the pixel/activation values of all feature maps laid end-to-end in a single row.

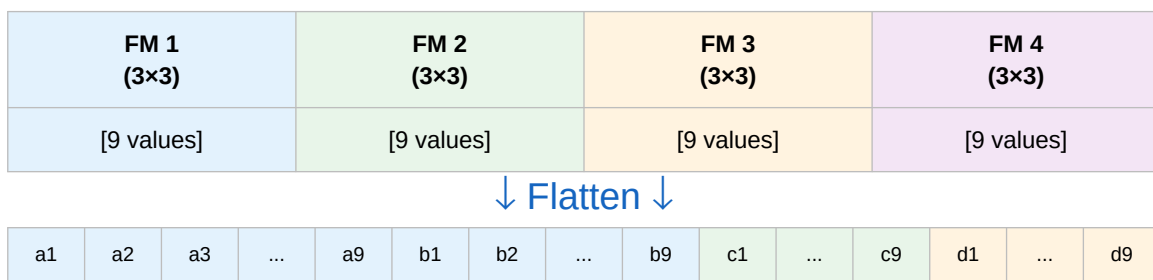


Fig: $3 \times 3 \times 4$ feature volume flattened to a 36-element 1-D vector

Why Flatten?

Convolutional layers output 3-D tensors; Dense (FC) layers require 1-D input.

Flattening is the bridge between the spatial feature extractor and the classifier.

No parameters are learned — it is a zero-cost reshaping step.

9.2 Dense (Fully Connected) Layer

A **Dense** (or Fully Connected) layer connects every input neuron to every output neuron. If the flattened vector has N values and the dense layer has M neurons, there are $N \times M$ weights plus M biases.

Output of neuron j :

$$z_j = \text{ReLU}(w_{j1} \cdot x_1 + w_{j2} \cdot x_2 + \dots + w_{jN} \cdot x_N + b_j)$$

Worked Example

Flattened vector ($N=4$): $x = [8, 3, 6, 5]$. Dense layer has **3 neurons**. Weights and biases:

Neuron	w_1	w_2	w_3	w_4	Bias	z (pre-ReLU)	After ReLU
$j=1$	0.2	0.5	-0.3	0.8	1.0	$0.2 \times 8 + 0.5 \times 3 + (-0.3) \times 6 + 0.8 \times 5 + 1.0 = 9.3$	9.3
$j=2$	-0.4	0.3	0.7	-0.2	0.5	$(-0.4) \times 8 + 0.3 \times 3 + 0.7 \times 6 + (-0.2) \times 5 + 0.5 = 1.6$	1.6

j=3	0.6	-0.1	0.4	0.3	-0.5	$0.6 \times 8 + (-0.1) \times 3 + 0.4 \times 6 + 0.3 \times 5 + (-0.5) = 8.8$	8.8
-----	-----	------	-----	-----	------	---	-----

Dense layer output vector: **[9.3, 1.6, 8.8]**

9.3 Softmax Output Layer

The final layer applies the **Softmax** function to convert raw scores (logits) into **class probabilities** that sum to 1. The class with the highest probability is the predicted label.

Softmax Formula:

$$P(\text{class } k) = \exp(z_k) / [\exp(z_1) + \exp(z_2) + \dots + \exp(z_K)]$$

Worked Example (continuing from above)

Dense output (logits): **z = [9.3, 1.6, 8.8]** Classes: Cat, Dog, Bird

Step	Cat (z=9.3)	Dog (z=1.6)	Bird (z=8.8)
exp(z)	exp(9.3) = 10938	exp(1.6) = 4.95	exp(8.8) = 6634
Sum of exp(z)	10938 + 4.95 + 6634 = 17577		
Softmax P	10938 / 17577 = 0.622	4.95 / 17577 = 0.000	6634 / 17577 = 0.377
Probability	62.2%	0.03%	37.7%

Prediction: **Cat** with 62.2% confidence. (Highest softmax probability wins.)

Softmax Properties

All output probabilities lie in (0, 1).

All probabilities sum to exactly 1.0.

The class with the highest probability is the final prediction.

During training, Cross-Entropy Loss compares the Softmax output to the true label.

9.4 Classification Head — Complete Flow

The table below shows the complete journey from the last pooling layer to the final class prediction:

Stage	Input Shape	Output Shape	Operation
Last Pool Layer	$H \times W \times D$	$H \times W \times D$	MaxPool / AvgPool
Flatten	$H \times W \times D$	$(H \times W \times D) \times 1$	Reshape to 1-D vector
Dense (Hidden)	$N \times 1$	$M \times 1$	$z = \text{ReLU}(Wx + b)$
Dense (Output)	$M \times 1$	$K \times 1$	Raw logits (one per class)
Softmax	$K \times 1$	$K \times 1$	Convert logits to probabilities
Prediction	K probs	1 label	$\text{argmax}(\text{probabilities})$

10. Complete End-to-End CNN Example

The diagram below traces a complete forward pass through a CNN classifier. An **8×8 grayscale image** is processed by two convolutional layers, two max-pooling layers, a flatten step, a dense (fully connected) layer, and finally a Softmax layer that outputs probabilities for three classes: **Cat, Dog and Bird**.

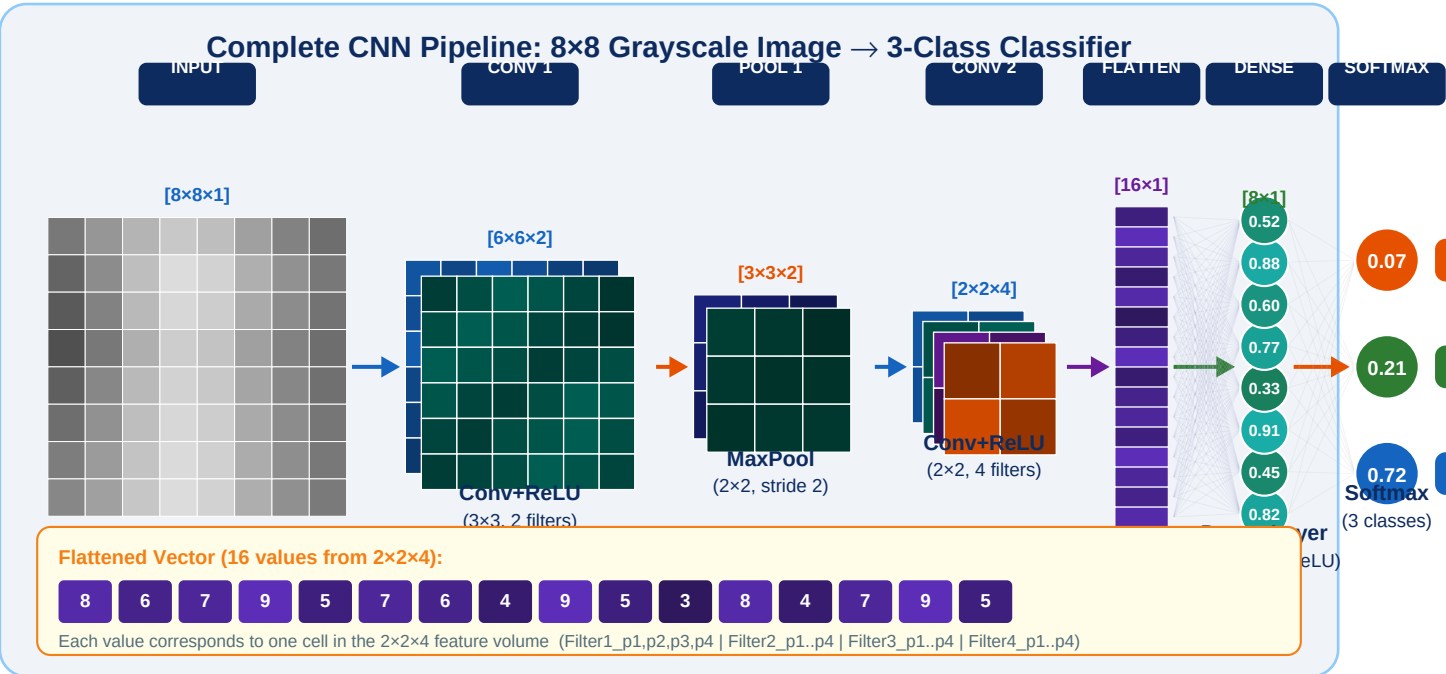


Fig: End-to-end CNN pipeline — 8×8 grayscale image classified into 3 categories. Predicted class: **Cat (72%)**.

Layer-by-Layer Breakdown

Layer	Input	Operation	Output	Notes
Input	8×8×1	—	8×8×1	Raw pixel values [0,255]
Conv 1	8×8×1	3×3 kernel, 2 filters, P=0, S=1	6×6×2	ReLU applied
MaxPool 1	6×6×2	2×2 pool, S=2	3×3×2	Spatial size halved
Conv 2	3×3×2	2×2 kernel, 4 filters, P=0, S=1	2×2×4	ReLU applied
Flatten	2×2×4	Reshape	16×1	2×2×4 = 16 values
Dense	16×1	FC: 16→8 neurons, ReLU	8×1	64 weights + 8 biases
Softmax	8×1	Logits → probabilities	3×1	3 class scores sum to 1
Output	3 probs	argmax	1 label	Cat: 72%, Dog: 21%, Bird: 7%

Prediction Result

Input: 8×8 grayscale image

After Conv1+ReLU: 6×6×2 feature maps (2 filters detect edges/patterns)

After MaxPool1: 3×3×2 (spatial downsampling by factor 2)

After Conv2+ReLU: $2 \times 2 \times 4$ feature maps (4 filters, richer features)

After Flatten: 16-element 1-D vector

After Dense+ReLU: 8-element activation vector

After Softmax: [Cat=0.72, Dog=0.21, Bird=0.07]

Final Prediction: CAT (highest probability: 72%)

11. Complete Summary

Component	Role	Key Formula / Note
Input Layer	Raw pixel tensor	$W \times H \times C$
Convolution	Spatial feature extraction	$O = (M - F + 2P) / S + 1$
Kernel/Filter	Learnable weight matrix	Size: $F \times F \times C$ per filter
Feature Map	Output of one filter	Encodes pattern presence
Bias	Learned offset	$y = w \cdot x + b$
Zero-Padding	Preserve border info	Adds zeros around edges
Stride	Step size of kernel	Larger $S \rightarrow$ smaller output
ReLU	Non-linear activation	$\max(0, x)$
Multiple Filters	Parallel feature detection	K filters $\rightarrow$ depth K
Flatten	Reshape 3-D $\rightarrow$ 1-D	$W \times H \times D$ values
Dense Layer	Fully connected classifier	$z = \text{ReLU}(Wx + b)$
Softmax	Probability distribution	$P(k) = \exp(z_k) / \sum \exp(z)$
Prediction	Highest probability class	$\text{argmax}(\text{Softmax output})$

CNN Pipeline at a Glance

Input Image

$\rightarrow$ [Conv + ReLU] $\rightarrow$ MaxPool

$\rightarrow$ [Conv + ReLU] $\rightarrow$ MaxPool

$\rightarrow$ Flatten

$\rightarrow$ Dense (FC) + ReLU

$\rightarrow$ Dense (Output)

$\rightarrow$ Softmax $\rightarrow$ Class Probabilities $\rightarrow$ Prediction